



**Bharat Ratna Indira Gandhi College of
Engineering, Solapur
Master Of Computer Application**

**Subject: Data Structure and Algorithms
(MCAC102)**

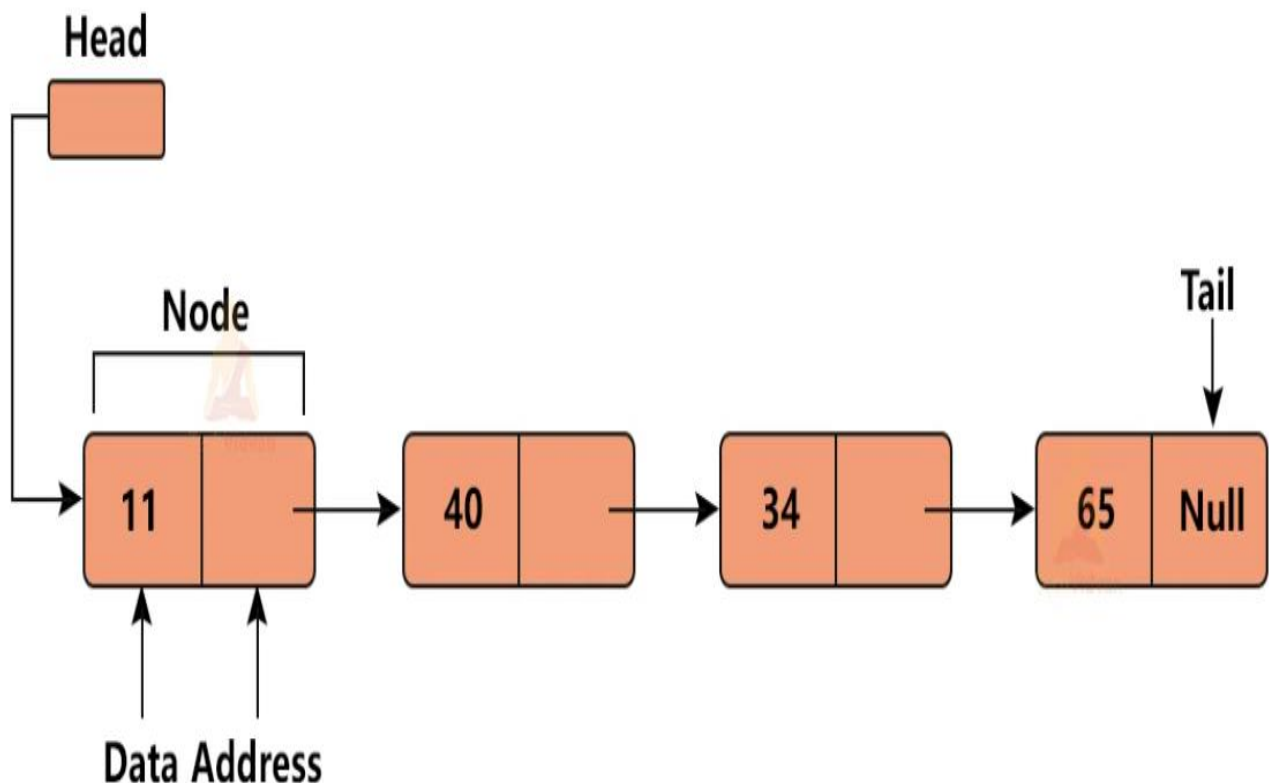
**Semester-I
Unit 3: Linked Lists**

❖ Linked List Data Structure

A **linked list** is a fundamental data structure in computer science. It mainly allows efficient **insertion** and **deletion** operations compared to **arrays**. Like arrays, it is also used to implement other data structures like stack, queue and deque.

➤ What is a Linked List?

A **linked list** is a linear data structure that consists of a series of nodes connected by pointers (in C or C++) or references (in Java, Python and JavaScript). Each node contains **data** and a **pointer/reference** to the next node in the list. Unlike **arrays**, **linked lists** allow for efficient **insertion** or **removal** of elements from any position in the list, as the nodes are not stored contiguously in memory.



Linked List vs Array:

Here's the comparison of Linked List vs Arrays

Linked List:

- i. **Data Structure:** Non-contiguous
- ii. **Memory Allocation:** Typically allocated one by one to individual elements
- iii. **Insertion/Deletion:** Efficient
- iv. **Access:** Sequential

Array:

- i. **Data Structure:** Contiguous
- ii. **Memory Allocation:** Typically allocated to the whole array
- iii. **Insertion/Deletion:** Inefficient
- iv. **Access:** Random

➤ Types of Linked List

1. Singly Linked List

A **singly linked list** is a fundamental data structure in computer science and programming. It is a collection of nodes where each node contains a **data field** and a **reference** (link) to the next node in the sequence. The last node in the list points to **null**, indicating the end of the list. This linear data structure allows for efficient insertion and deletion operations, making it a popular choice for various applications.

2. Doubly Linked List

A **doubly linked list** is a data structure that consists of a set of nodes, each of which contains a **value** and **two pointers**, one pointing to the **previous node** in the list and one pointing to the **next node** in the list. This allows for efficient traversal of the list in **both directions**, making it suitable for applications where frequent **insertions** and **deletions** are required.

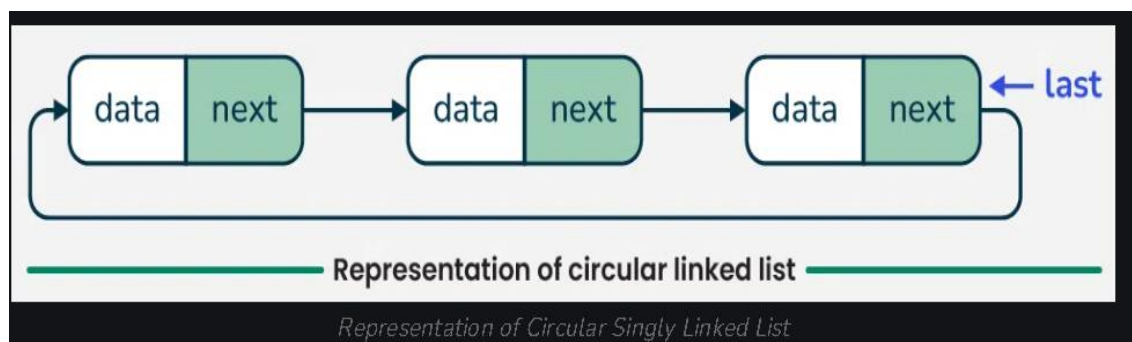
3. Circular Linked List

A **circular linked list** is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to **NULL**, the last node in a circular linked list points back to the first node. This means that you can keep traversing the list without ever reaching a **NULL** value.

Types of Circular Linked Lists

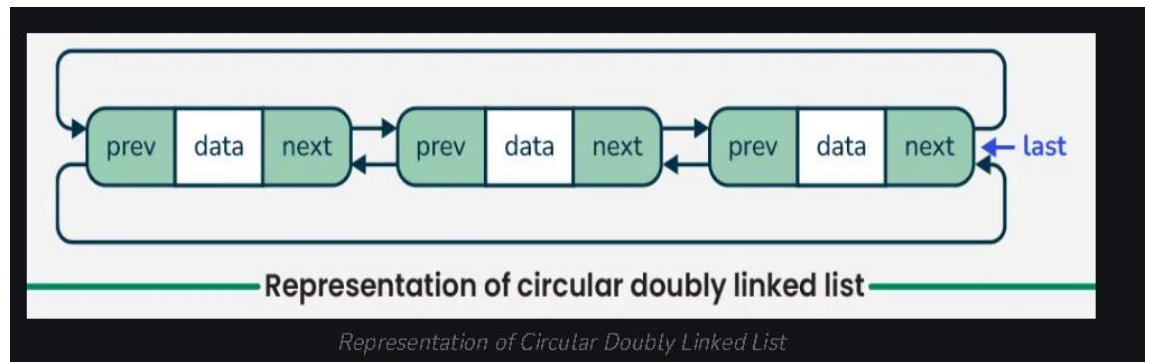
A. Circular Singly Linked List

In **Circular Singly Linked List**, each node has just one pointer called the “**next**” pointer. The next pointer of **last node** points back to the **first node** and this results in forming a circle. In this type of Linked list, we can only move through the list in one direction.



B. Circular Doubly Linked List

In **circular doubly linked list**, each node has two pointers **prev** and **next**, similar to doubly linked list. The **prev** pointer points to the previous node and the **next** points to the next node. Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



4. Circular Doubly Linked List

✚ Characteristics of Circular Doubly Linked List:

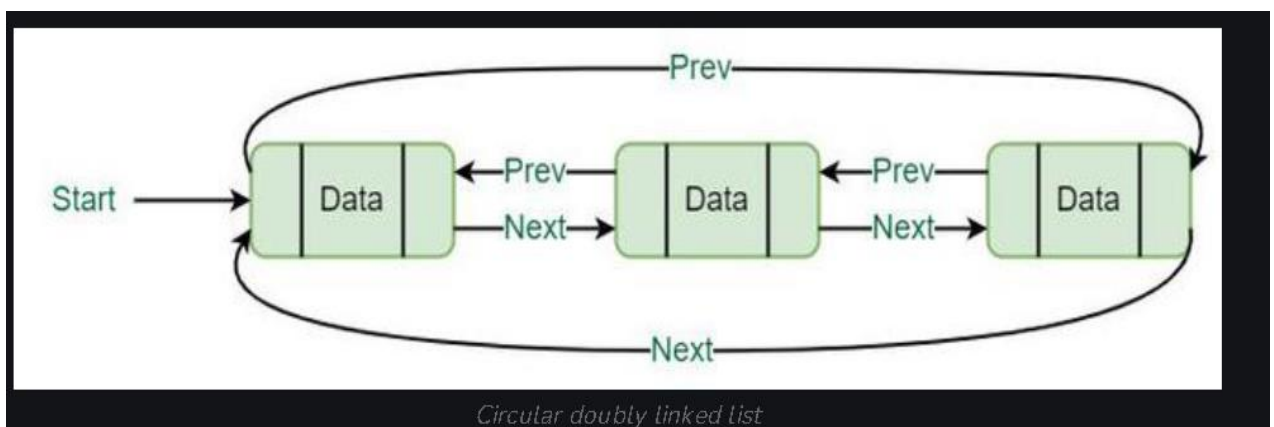
A circular doubly linked list has the following properties:

- i. **Circular:** A circular doubly linked list's main feature is that it is circular in design.
- ii. **Doubly Linked:** Each node in a circular doubly linked list has two pointers – one pointing to the node before it and the other pointing to the node after it.
- iii. **Header Node:** At the start of circular doubly linked lists, a header node or sentinel node is frequently used. This node is used to make the execution of certain operations on the list simpler even though it is not a component of the list's actual contents.

Applications of Circular Doubly Linked List:

Circular doubly linked lists are used in a variety of applications, some of which include:

- i. **Implementation of Circular Data Structures:** Circular doubly linked lists are extremely helpful in the construction of circular data structures like circular queues and circular buffers, which are both circular in nature.
- ii. **Implementing Undo-Redo Operations:** Text editors and other software programs can use circular doubly linked lists to implement undo-redo operations.
- iii. **Music Player Playlist:** Playlists in music players are frequently implemented using circular doubly linked lists. Each song is kept as a node in the list in this scenario, and the list can be circled to play the songs in the order they are listed.
- iv. **Cache Memory Management:** To maintain track of the most recently used cache blocks, circular doubly linked lists are employed in cache memory management.



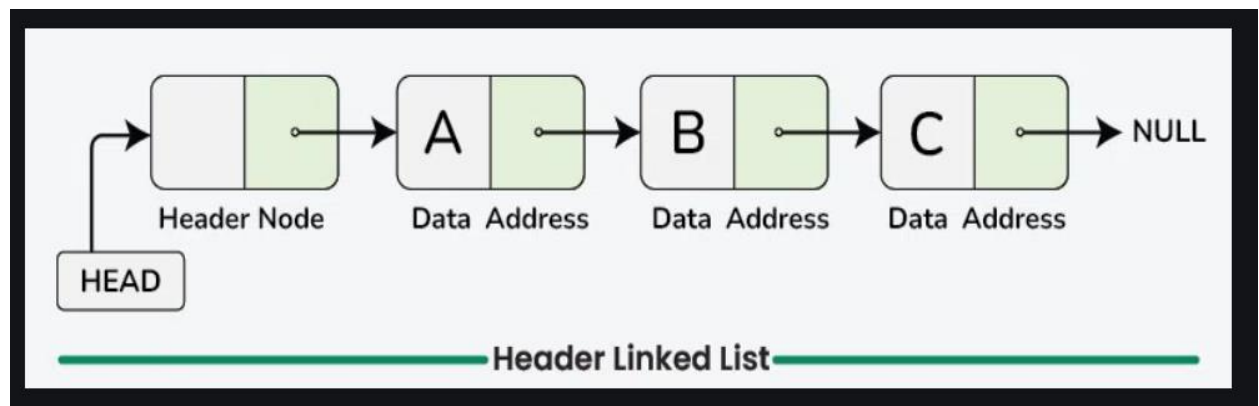
5. Header Linked List

A header linked list is a special type of linked list which consists of a **header** node in the **beginning** of the list. The header node can store **meta data** about the linked list. This type of list is useful when information other than that found in each node is needed. For example, suppose there is an application in which the **number of items** in a list is often **calculated**. Usually, a list is always traversed to **find the length** of the list. However, information can be easily obtained if the **current length** is maintained in an additional header node.

🌈 Types of Header Linked List

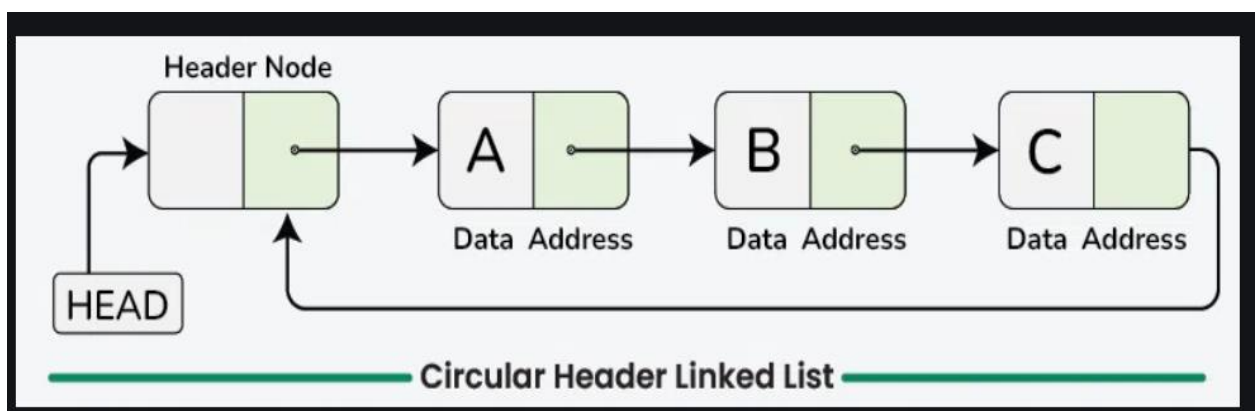
A. Singly or Doubly Header Linked List

It is a list whose last node contains the **NULL** pointer. In the **header** linked list the start pointer always points to the **header** node. **start -> next = NULL** indicates that the header linked list is empty. The operations that are possible on this type of linked list are **Insertion**, **Deletion**, and **Traversing**.



B. Singly or Doubly Circular Header Linked List

A list in which the **last** node points **back** to the **header** node is called **circular header linked list**. The chains do not indicate **first or last** nodes. In this case, external pointers provide a frame of reference because last node of a circular linked list does not contain the **NULL** pointer. The possible operations on this type of linked list are **Insertion, Deletion** and **Traversing**.



❖ Linked Organization Representation of Linked List in Memory Singly

A **singly linked list** is a type of data structure where each element (or node) stores data and has a pointer (or reference) to the next node in the sequence. Unlike arrays, which use contiguous memory locations, the nodes of a linked list are distributed across memory, and each node knows where the next node is located through the use of a pointer. This allows for efficient insertion and deletion operations but requires extra memory to store pointers.

➤ **Structure of a Node**

In a singly linked list, each node typically contains two parts:

1. **Data:** This stores the actual value.
2. **Next Pointer:** This is a reference or pointer to the next node in the linked list. The last node's pointer will point to NULL, indicating the end of the list.

```
struct Node {  
  
    int data;    // Holds the data  
  
    struct Node* next; // Holds the address of the next node  
  
};
```

➤ **Memory Representation of Singly Linked List**

The nodes of a singly linked list are not stored in contiguous memory locations. Each node is scattered in memory and connected using the next pointer.

Consider a memory representation where each box represents a node, containing the data value and the memory address (pointer) to the next node:

[Data: 10 | Next: 0x2004] -> [Data: 20 | Next: 0x3008] -> [Data: 30 | Next: 0xNULL]

- i. The first node stores the value 10, and its next pointer points to the memory address 0x2004, which contains the second node.
- ii. The second node stores the value 20, and its next pointer points to 0x3008, which contains the third node.
- iii. The third node stores the value 30, and its next pointer is NULL, indicating the end of the list.

Example

Let's build a singly linked list with the following data values: 10 -> 20 -> 30.

1. Create Nodes in Memory:

- Each node is dynamically allocated, so they are scattered in memory.
- For instance:
 - Node 1 (stores 10) is at memory location 0x1004
 - Node 2 (stores 20) is at memory location 0x2008
 - Node 3 (stores 30) is at memory location 0x300C

2. Visual Representation of Memory:

Node 1 at address 0x1004:

[Data: 10 | Next: 0x2008] --> Points to Node 2

Node 2 at address 0x2008:

[Data: 20 | Next: 0x300C] --> Points to Node 3

Node 3 at address 0x300C:

[Data: 30 | Next: NULL] --> NULL indicates the end of the list

- i. **Node 1** contains data = 10 and a pointer to the next node, located at address 0x2008.
- ii. **Node 2** contains data = 20 and a pointer to the next node, located at address 0x300C.
- iii. **Node 3** contains data = 30 and its next pointer is NULL, meaning it is the last node.

Inserting and Traversing

i. Inserting a New Node:

To insert a node after node 1 (let's say with value 15), you:

1. Create a new node at some memory address (e.g., 0x4000).
2. Set the next pointer of the new node to point to the node that follows node 1 (0x2008).
3. Update the next pointer of node 1 to point to the new node.

Memory representation after insertion:

Node 1: [Data: 10 | Next: 0x4000]

Node 2: [Data: 15 | Next: 0x2008]

Node 3: [Data: 20 | Next: 0x300C]

Node 4: [Data: 30 | Next: NULL]

ii. Traversal:

To traverse a singly linked list, you start at the head node and follow the next pointers until you reach a node where `next == NULL`.

Example traversal pseudocode:

```
void traverse(struct Node* head) {  
  
    struct Node* current = head;
```

```
while (current != NULL) {  
  
    printf("%d -> ", current->data);  
  
    current = current->next;  
  
}  
  
printf("NULL\n");  
  
}
```

Advantages and Disadvantages of Singly Linked List

- **Advantages:**

- I. Dynamic size:** You can easily grow or shrink the list by adding/removing nodes without worrying about contiguous memory.
- II. Efficient insertion/deletion:** Adding or removing elements, especially from the beginning or middle, is more efficient than an array, which requires shifting elements.

- **Disadvantages:**

- I. Extra memory for pointers:** Every node requires additional memory to store the next pointer.
- II. Sequential access:** Unlike arrays, random access (e.g., accessing the i-th element) is inefficient as you must traverse the list from the beginning.

➤ **Representation of a Doubly Linked List in Memory**

A **doubly linked list** is a more complex variation of the singly linked list. Each node in a doubly linked list stores references to both the **next** node and the **previous** node. This enables traversal in both directions (forward and backward) through the list, which adds flexibility but also increases the memory usage compared to a singly linked list.

Structure of a Node

In a doubly linked list, each node contains three parts:

1. **Data:** The actual data stored in the node.
2. **Next Pointer:** A pointer (reference) to the next node in the sequence.
3. **Previous Pointer:** A pointer (reference) to the previous node in the sequence.

Node Structure (in C-like pseudocode):

```
struct Node {  
  
    int data;      // Holds the data  
  
    struct Node* next; // Holds the address of the next node  
  
    struct Node* prev; // Holds the address of the previous node  
  
};
```

Sem-I[DSA]-Unit (3)

The nodes of a doubly linked list are also scattered across memory, similar to a singly linked list. However, each node has two pointers:

1. One pointing to the **next** node.
2. One pointing to the **previous** node.

Consider the following diagram to illustrate the memory representation:

NULL <- [Prev: NULL | Data: 10 | Next: 0x2008] <->

[Prev: 0x1004 | Data: 20 | Next: 0x300C] <->

[Prev: 0x2008 | Data: 30 | Next: NULL] -> NULL

(all are in same line)

- a. The first node stores 10, with the prev pointer pointing to NULL (since it's the first node) and the next pointer pointing to the memory address 0x2008 where the second node is stored.
- b. The second node stores 20, with the prev pointer pointing to 0x1004 (address of the first node) and the next pointer pointing to 0x300C (address of the third node).
- c. The third node stores 30, with the prev pointer pointing to 0x2008 (address of the second node) and the next pointer pointing to NULL (since it's the last node).

Example

Let's build a doubly linked list with the following data values: 10 <-> 20 <-> 30.

1. Create Nodes in Memory:

- Each node is dynamically allocated and distributed in memory. For instance:
 - a. Node 1 (stores 10) is at memory location 0x1004.
 - b. Node 2 (stores 20) is at memory location 0x2008.
 - c. Node 3 (stores 30) is at memory location 0x300C.

2. Visual Representation of Memory:

- i. The prev pointer of the first node points to NULL because it has no previous node.
- ii. The next pointer of the last node points to NULL because it has no next node.

Node 1 at address 0x1004:

[Prev: NULL | Data: 10 | Next: 0x2008] --> Points to Node 2

Node 2 at address 0x2008:

[Prev: 0x1004 | Data: 20 | Next: 0x300C] --> Points to Node 3

Node 3 at address 0x300C:

[Prev: 0x2008 | Data: 30 | Next: NULL] --> End of the list

➤ **Insertion and Traversal in a Doubly Linked List**

1.Insertion

Let's say we want to insert a new node with the value 15 between 10 and 20. The steps are:

- i. Create a new node (e.g., at memory location 0x4000) with data = 15.
- ii. Update the next pointer of node 1 (which stores 10) to point to the new node (instead of node 2).
- iii. Update the prev pointer of node 2 (which stores 20) to point to the new node (instead of node 1).
- iv. Set the next pointer of the new node to point to node 2, and its prev pointer to point to node 1.

After insertion, the memory layout will look like this:

Node 1 at address 0x1004:

[Prev: NULL | Data: 10 | Next: 0x4000] --> Points to Node 4

Node 4 at address 0x4000:

[Prev: 0x1004 | Data: 15 | Next: 0x2008] --> Points to Node 2

Node 2 at address 0x2008:

[Prev: 0x4000 | Data: 20 | Next: 0x300C] --> Points to Node 3

Node 3 at address 0x300C:

[Prev: 0x2008 | Data: 30 | Next: NULL] --> End of the list

2.Traversal

Traversal in a doubly linked list can be done in two directions:

- i. **Forward Traversal:** Start from the head (first node) and follow the next pointers.
- ii. **Backward Traversal:** Start from the tail (last node) and follow the prev pointers.

A. Forward Traversal Pseudocode:

```
void forward_traverse(struct Node* head) {  
  
    struct Node* current = head;  
  
    while (current != NULL) {  
  
        printf("%d -> ", current->data);  
  
        current = current->next;  
  
    }  
  
    printf("NULL\n");  
  
}
```

B. Backward Traversal Pseudocode:

```
void backward_traverse(struct Node* tail){  
  
    struct Node* current = tail;  
  
    while (current != NULL){  
  
        printf("%d -> ", current->data);  
  
        current = current->prev;  
  
    }  
  
    printf("NULL\n");  
  
}
```

If the list contains 10 <-> 15 <-> 20 <-> 30, the output for forward traversal would be:

10 -> 15 -> 20 -> 30 -> NULL

For backward traversal, assuming you start from the tail (node with 30):

30 -> 20 -> 15 -> 10 -> NULL

➤ **Advantages and Disadvantages of Doubly Linked List**

Advantages:

1. Two-way traversal: You can traverse the list in both forward and backward directions using the next and prev pointers.

2. Efficient deletion: Unlike a singly linked list, where you need access to the previous node to delete a node, the prev pointer makes deletion from any part of the list more straightforward.
3. More flexible insertion: Since you can traverse backward, inserting elements after a node or before a node becomes easier.

Disadvantages:

1. Extra memory overhead: Every node in a doubly linked list uses extra memory for storing the prev pointer, which can be significant for large lists.
2. Complexity: Maintaining two pointers (prev and next) increases the complexity of the list operations (e.g., during insertion, deletion, etc.).
3. Slower operations: Because each node contains more information (two pointers), some operations like insertion, especially in large lists, can be slightly slower due to the increased need to manage both pointers.
- 4.

➤ **Representation of a Circular Linked List in Memory**

A **circular linked list** is a variation of the linked list where the last node's pointer points back to the first node, forming a circular structure. This can be implemented as either a **singly circular linked list** (where each node points to the next node and the last node points to the first) or a **doubly circular linked list** (where each node points to both the previous

and next nodes, and the last node's next pointer points to the first node while the first node's previous pointer points to the last node).

Structure of a Node in a Circular Singly Linked List

Each node in a circular singly linked list contains two parts:

- a. **Data:** The actual value stored in the node.
- b. **Next Pointer:** A pointer that points to the next node in the sequence.

Unlike a traditional singly linked list, the next pointer of the last node points to the **first node**, not NULL.

Node Structure (in C-like pseudocode):

```
struct Node {  
  
    int data;      // Holds the data  
  
    struct Node* next; // Holds the address of the next node  
  
};
```

Memory Representation of a Circular Singly Linked List

In a circular singly linked list, the last node's next pointer does not point to NULL but instead points back to the first node in the list, creating a loop.

Consider the following circular singly linked list with three nodes: 10 -> 20 -> 30, where the last node (which stores 30) points back to the first node (which stores 10).

Visual Representation:

- **Memory Representation of a Circular Singly Linked List**

In a circular singly linked list, the last node's next pointer does not point to NULL but instead points back to the first node in the list, creating a loop.

Consider the following circular singly linked list with three nodes: 10 -> 20 -> 30, where the last node (which stores 30) points back to the first node (which stores 10).

Visual Representation:

[Data: 10 | Next: 0x2008] -> [Data: 20 | Next: 0x300C] -> [Data: 30 | Next: 0x1004]



- Node 1** (with data 10) has its next pointer pointing to node 2.
- Node 2** (with data 20) has its next pointer pointing to node 3.
- Node 3** (with data 30) has its next pointer pointing back to node 1, forming a circular loop.

Example

Let's build a circular singly linked list with the values 10 -> 20 -> 30.

1. **Create Nodes in Memory:**

- The nodes are dynamically allocated in memory. For instance:
 - a. Node 1 (stores 10) is at memory location 0x1004.

- b. Node 2 (stores 20) is at memory location 0x2008.
- c. Node 3 (stores 30) is at memory location 0x300C.

2. Memory Representation:

Node 1 at address 0x1004:

[Data: 10 | Next: 0x2008] --> Points to Node 2

Node 2 at address 0x2008:

[Data: 20 | Next: 0x300C] --> Points to Node 3

Node 3 at address 0x300C:

[Data: 30 | Next: 0x1004] --> Points back to Node 1 (circular link)

- **Insertion and Traversal in a Circular Singly Linked List**

1.Insertion

Let's say we want to insert a new node with the value 25 between 20 and 30.

Steps for insertion:

1. Create a new node (e.g., at memory location 0x4000) with data = 25.
2. Set the next pointer of node 2 (which stores 20) to point to the new node (instead of node 3).
3. Set the next pointer of the new node to point to node 3 (which stores 30).

After insertion, the memory layout will look like this:

Node 1 at address 0x1004:

[Data: 10 | Next: 0x2008]

Node 2 at address 0x2008:

[Data: 20 | Next: 0x4000]

Node 4 at address 0x4000:

[Data: 25 | Next: 0x300C]

Node 3 at address 0x300C:

[Data: 30 | Next: 0x1004] --> Points back to Node 1

2.Traversal

In a circular singly linked list, traversal is similar to a regular singly linked list except that when we reach the last node, we continue from the first node again (since the list is circular).

Pseudocode for traversal of a circular singly linked list:

```
void traverse(struct Node* head) {  
    if (head == NULL) return;  
    struct Node* current = head;  
    do {  
        printf("%d -> ", current->data);  
        current = current->next;  
    } while (current != head); // Loop until we return to the head node
```

```
printf("HEAD\n"); // Indicate that we've completed the loop  
}
```

For example, if the list contains 10 -> 20 -> 25 -> 30, the output of the traversal would be:

10 -> 20 -> 25 -> 30 -> HEAD

Structure of a Node in a Circular Doubly Linked List

In a **circular doubly linked list**, each node contains three parts:

1. **Data:** The actual value stored in the node.
2. **Next Pointer:** A pointer to the next node in the sequence.
3. **Previous Pointer:** A pointer to the previous node in the sequence.

In a circular doubly linked list, the next pointer of the last node points to the first node, and the prev pointer of the first node points to the last node, creating a circular structure in both directions.

Node Structure (in C-like pseudocode):

```
struct Node {  
  
    int data;      // Holds the data  
  
    struct Node* next; // Holds the address of the next node  
  
    struct Node* prev; // Holds the address of the previous node  
  
};
```


➤ **Memory Representation of a Circular Doubly Linked List**

Consider the following circular doubly linked list with three nodes: 10 <-> 20 <-> 30,
where:

- i. The next pointer of the last node (which stores 30) points back to the first node (which stores 10), and
- ii. The prev pointer of the first node points to the last node (forming a loop).
- iii. **Node 1** (with data 10) has its `next` pointer pointing to node 2 and its `prev` pointer pointing to node 3.
- iv. **Node 2** (with data 20) has its `next` pointer pointing to node 3 and its `prev` pointer pointing to node 1.
- v. **Node 3** (with data 30) has its `next` pointer pointing back to node 1 and its `prev` pointer pointing to node 2, forming a circular doubly linked list.

Advantages and Disadvantages of Circular Linked List

Advantages:

- i. **No NULL nodes:** There is no need to check for NULL in traversal, as you can always continue from the last node back to the first.

- ii. **Efficient cycling:** The circular structure makes it easy to implement algorithms that require wrapping around the list, such as round-robin scheduling.
- iii. **Flexible traversal:** In a circular doubly linked list, traversal can easily happen in both directions (forward and backward).

Disadvantages:

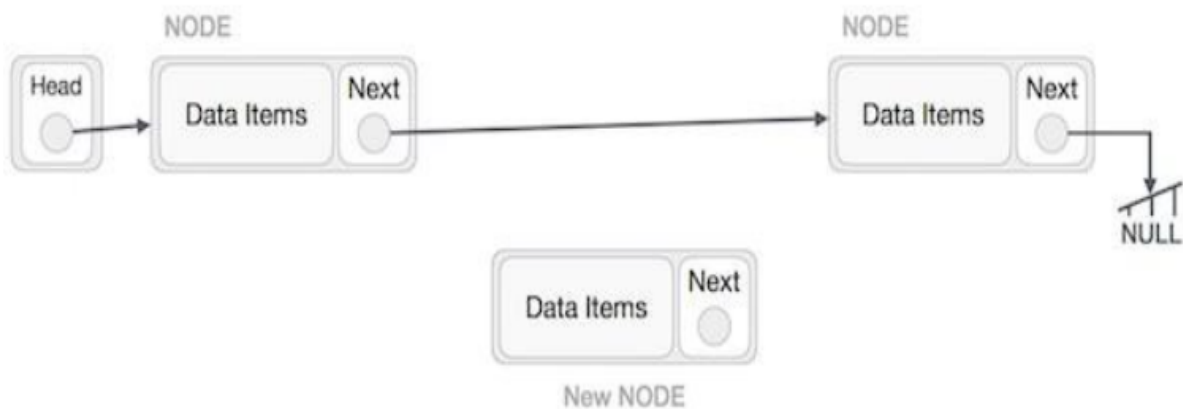
- i. **More complex implementation:** Maintaining the circular structure (especially in the case of a doubly linked list) requires extra care when performing insertions and deletions.
- ii. **Extra memory overhead:** For circular doubly linked lists, you have to store both next and prev pointers, which increases memory usage.

❖ **Operations on Singly Linked List**

- i. **Insertion** – Adds an element at the beginning of the list.
- ii. **Deletion** – Deletes an element at the beginning of the list.
- iii. **Display** – Displays the complete list.
- iv. **Search** – Searches an element using the given key.
- v. **Delete** – Deletes an element using the given key.

1. Insertion Operation

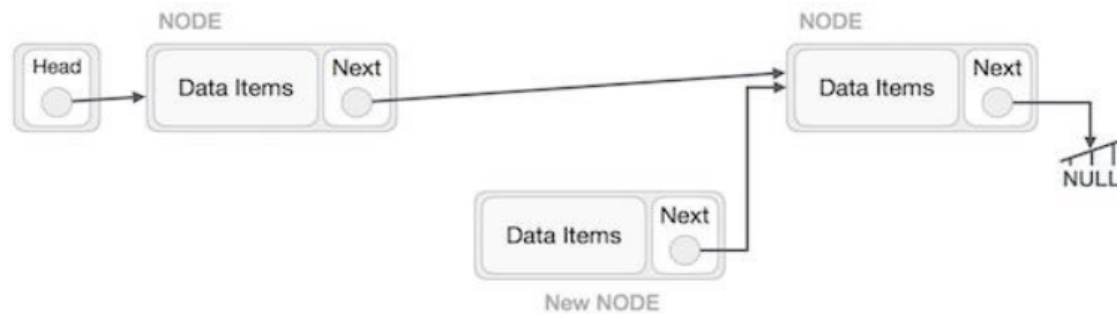
Adding a new node in linked list is a more than one step activity. We shall learn this with diagrams here. First, create a node using the same structure and find the location where it has to be inserted.



Imagine that we are inserting a node B (NewNode), between A (LeftNode) and C (RightNode). Then point B.next to C –

```
NewNode.next -> RightNode;
```

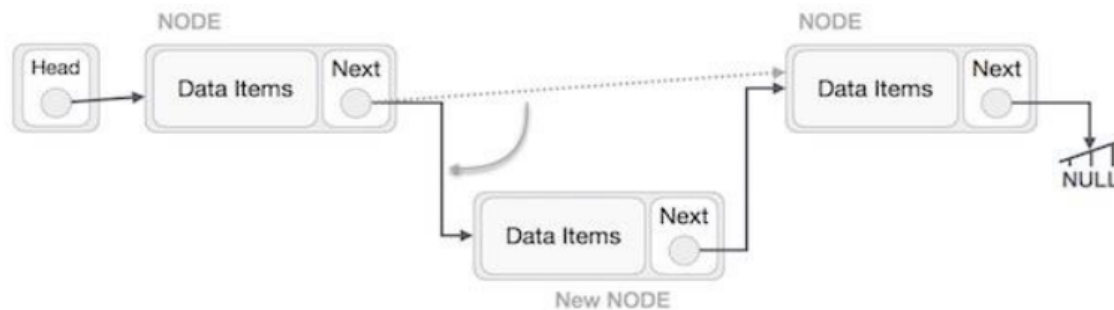
It should look like this –



Now, the next node at the left should point to the new node.

```
LeftNode.next -> NewNode;
```

This will put the new node in the middle of the two. The new list should look like this –



Algorithm

1. START
2. Create a node to store the data
3. Check if the list is empty
4. If the list is empty, add the data to the node and assign the head pointer to it.
5. If the list is not empty, add the data to a node and link to the current head. Assign the head to the newly added node.
6. END

Example:

```
#include <bits/stdc++.h>
```

```
#include <string>
```

```
using namespace std;
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *head = NULL;
```

```
struct node *current = NULL;
```

```
// display the list
```

```
void printList(){
```

```
    struct node *p = head;
```

```
    cout << "\n[";
```

```
    //start from the beginning
```

```
    while(p != NULL) {
```

```
        cout << " " << p->data << " ";
```

```
        p = p->next;
```

```
    }

    cout << "]\n";

}

//insertion at the beginning

void insertatbegin(int data){

    //create a link

    struct node *lk = (struct node*) malloc(sizeof(struct node));

    lk->data = data;

    // point it to old first node

    lk->next = head;

    //point first to new first node

    head = lk;

}

int main(){

    insertatbegin(12);

    insertatbegin(22);

    insertatbegin(30);

    insertatbegin(44);
```

```
insertatbegin(50);

cout << "Linked List: ";

// print list

printList();

}
```

Output:

Linked List: [12 22 30 44 50]

2. Delete Operation

a. Deletion At Beginning

Algorithm

```
1. START
2. Assign the head pointer to the next node in the list
3. END
```

Example:

```
#include <bits/stdc++.h>

#include <string>

using namespace std;

struct node {
```

```
int data;

struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

// display the list

void printList(){

    struct node *p = head;

    cout << "\n[";

    //start from the beginning

    while(p != NULL) {

        cout << " " << p->data << " ";

        p = p->next;

    }

    cout << "]\n";

}

//insertion at the beginning

void insertatbegin(int data){
```



```
//create a link

struct node *lk = (struct node*) malloc(sizeof(struct node));

lk->data = data;

// point it to old first node

lk->next = head;

//point first to new first node

head = lk;

}

void deleteatbegin(){

    head = head->next;

}

int main(){

    insertatbegin(12);

    insertatbegin(22);

    insertatbegin(30);

    insertatbegin(44);

    insertatbegin(50);

    cout << "Linked List: ";
```

```
// print list

printList();

deleteatbegin();

cout << "\nLinked List after deletion: ";

printList();

}
```

Output

```
Linked List:
[ 50  44  30  22  12 ]
Linked List after deletion:
[ 44  30  22  12 ]
```

b. Deletion At Ending

Algorithm

1. START
2. Iterate until you find the second last element in the list.
3. Assign NULL to the second last element in the list.
4. END

```
#include <bits/stdc++.h>
```

```
#include <string>
```

```
using namespace std;
```

```
struct node {  
  
    int data;  
  
    struct node *next;  
  
};  
  
struct node *head = NULL;  
  
struct node *current = NULL;  
  
// Displaying the list  
  
void printList(){  
  
    struct node *p = head;  
  
    while(p != NULL) {  
  
        cout << " " << p->data << " ";  
  
        p = p->next;  
  
    }  
  
}  
  
// Insertion at the beginning  
  
void insertatbegin(int data){  
  
    //create a link  
  
    struct node *lk = (struct node*) malloc(sizeof(struct node));
```

```
lk->data = data;

// point it to old first node

lk->next = head;

//point first to new first node

head = lk;

}

void deleteatend(){

    struct node *linkedList = head;

    while (linkedList->next->next != NULL)

        linkedlist = linkedlist->next;

    linkedlist->next = NULL;

}

int main(){

    insertatbegin(12);

    insertatbegin(22);

    insertatbegin(30);

    insertatbegin(44);

    insertatbegin(50);
```

```
cout << "Linked List: ";

// print list

printList();

deleteatend();

cout << "\nLinked List after deletion: ";

printList();

}
```

Output

```
Linked List:
[ 55  40  30  22  12 ]
Linked List after deletion:
[ 55  40  30  22 ]
```

c. Deletion At Given Position

Algorithm

1. START
2. Iterate until find the current node at position in the list.
3. Assign the adjacent node of current node in the list to its previous node.
4. END

```
#include <bits/stdc++.h>

#include <string>

using namespace std;

struct node {

    int data;

    struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

// display the list

void printList(){

    struct node *p = head;

    cout << "\n[";

    //start from the beginning

    while(p != NULL) {

        cout << " " << p->data << " ";

        p = p->next;

    }
```

```
        cout << "]\n";
    }

    //insertion at the beginning

    void insertatbegin(int data){

        //create a link

        struct node *lk = (struct node*) malloc(sizeof(struct node));

        lk->data = data;

        // point it to old first node

        lk->next = head;

        //point first to new first node

        head = lk;

    }

    void deletenode(int key){

        struct node *temp = head, *prev;

        if (temp != NULL && temp->data == key) {

            head = temp->next;

            return;

        }

    }
```

```
// Find the key to be deleted

while (temp != NULL && temp->data != key) {

    prev = temp;

    temp = temp->next;

}

// If the key is not present

if (temp == NULL) return;

// Remove the node

prev->next = temp->next;

}

int main(){

    insertatbegin(12);

    insertatbegin(22);

    insertatbegin(30);

    insertatbegin(44);

    insertatbegin(50);

    cout << "Linked List: ";

    // print list
```



```
printList();

deletenode(30);

cout << "\nLinked List after deletion: ";

printList();

}
```

Output

```
Linked List:
[ 55 40 30 22 12 ]
Linked List after deletion:
[ 55 40 22 12 ]
```

3. Reversal Operation

Algorithm

Step by step process to reverse a linked list is as follows –

1. START
2. We use three pointers to perform the reversing:
prev, next, head.
3. Point the current node to head and assign its next value to
the prev node.
4. Iteratively repeat the step 3 for all the nodes in the list.
5. Assign head to the prev node.

```
#include <bits/stdc++.h>

struct node {

    int data;

    struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

// display the list

void printList(){

    struct node *p = head;

    printf("\n[");

    //start from the beginning

    while(p != NULL) {

        printf(" %d ",p->data);

        p = p->next;

    }

    printf("]");

}
```

//insertion at the beginning

void insertatbegin(int data){

//create a link

struct node *lk = (struct node*) malloc(sizeof(struct node));

lk->data = data;

// point it to old first node

lk->next = head;

//point first to new first node

head = lk;

}

void reverseList(struct node head){**

struct node *prev = NULL, *cur=*head, *tmp;

while(cur!= NULL) {

tmp = cur->next;

cur->next = prev;

prev = cur;

cur = tmp;

}

```
        *head = prev;

    }

    int main(){

        int k=0;

        insertatbegin(12);

        insertatbegin(22);

        insertatbegin(30);

        insertatbegin(40);

        insertatbegin(55);

        printf("Linked List: ");

        // print list

        printList();

        reverseList(&head);

        printf("\nReversed Linked List: ");

        printList();

        return 0;

    }
```

Output

```
Linked List:  
[ 55 40 30 22 12 ]  
Reversed Linked List:  
[ 12 22 30 40 55 ]
```

4. Search Operation

Algorithm

```
1 START  
2 If the list is not empty, iteratively check if the list  
  contains the key  
3 If the key element is not present in the list, unsuccessful  
  search  
4 END
```

```
#include <bits/stdc++.h>
```

```
#include <string>
```

```
using namespace std;
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *head = NULL;
```

```
struct node *current = NULL;

// display the list

void printList(){

    struct node *p = head;

    cout << "\n[";

    //start from the beginning

    while(p != NULL) {

        cout << " " << p->data << " ";

        p = p->next;

    }

    cout << "];"

}

//insertion at the beginning

void insertatbegin(int data){

    //create a link

    struct node *lk = (struct node*) malloc(sizeof(struct node));

    lk->data = data;
```

```
// point it to old first node
```

```
lk->next = head;
```

```
//point first to new first node
```

```
head = lk;
```

```
}
```

```
int searchlist(int key){
```

```
    struct node *temp = head;
```

```
    while(temp != NULL) {
```

```
        if (temp->data == key) {
```

```
            return 1;
```

```
        }
```

```
        temp=temp->next;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int main(){
```

```
    int k = 0;
```

```
    insertatbegin(12);
```

```
insertatbegin(22);

insertatbegin(30);

insertatbegin(44);

insertatbegin(50);

cout << "Linked List: ";

// print list

printList();

int ele = 16;

cout<<"\nElement to be searched is: "<<ele;

k = searchlist(ele);

if (k == 1)

    cout << "\nElement is found";

else

    cout << "\nElement is not found in the list";

}
```

Output

```
Linked List:
[ 55 40 30 22 12 ]
Element to be searched is: 30
Element is found
```


5. Traversal Operation

Algorithm

```
1. START
2. While the list is not empty and did not reach the end of the list,
   print the data in each node
3. END
```

```
#include <bits/stdc++.h>
```

```
#include <string>
```

```
using namespace std;
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *head = NULL;
```

```
struct node *current = NULL;
```

```
// Displaying the list
```

```
void printList(){
```

```
struct node *p = head;

while(p != NULL) {

    cout << " " << p->data << " ";

    p = p->next;

}

}
```

// Insertion at the beginning

```
void insertatbegin(int data){
```

```
    //create a link
```

```
    struct node *lk = (struct node*) malloc(sizeof(struct node));
```

```
    lk->data = data;
```

```
    // point it to old first node
```

```
    lk->next = head;
```

```
    //point first to new first node
```

```
    head = lk;
```

```
    }  
  
    int main(){  
  
        insertatbegin(12);  
  
        insertatbegin(22);  
  
        insertatbegin(30);  
  
        insertatbegin(44);  
  
        insertatbegin(50);  
  
        cout << "Linked List: ";  
  
        // print list  
  
        printList();  
  
    }
```

Output

```
Linked List: 50 44 30 22 12
```

6. Complete Implementation

```
#include <bits/stdc++.h>

#include <string>

using namespace std;

struct node {

    int data;

    struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

// display the list

void printList(){

    struct node *p = head;

    cout << "\n[";

    //start from the beginning

    while(p != NULL) {

        cout << " " << p->data << " ";

        p = p->next;
```

```
    }

    cout << "]\n";

}

//insertion at the beginning

void insertatbegin(int data){

    //create a link

    struct node *lk = (struct node*) malloc(sizeof(struct node));

    lk->data = data;

    // point it to old first node

    lk->next = head;

    //point first to new first node

    head = lk;

}

void insertatend(int data){

    //create a link

    struct node *lk = (struct node*) malloc(sizeof(struct node));

    lk->data = data;

    struct node *linkedlist = head;
```

```
// point it to old first node

while(linkedlist->next != NULL)

    linkedlist = linkedlist->next;

//point first to new first node

linkedlist->next = lk;

}

void insertafternode(struct node *list, int data){

    struct node *lk = (struct node*) malloc(sizeof(struct node));

    lk->data = data;

    lk->next = list->next;

    list->next = lk;

}

void deleteatbegin(){

    head = head->next;

}

void deleteatend(){

    struct node *linkedlist = head;

    while (linkedlist->next->next != NULL)
```

```
        linkedlist = linkedlist->next;

        linkedlist->next = NULL;

    }

    void deletenode(int key){

        struct node *temp = head, *prev;

        if (temp != NULL && temp->data == key) {

            head = temp->next;

            return;

        }

        // Find the key to be deleted

        while (temp != NULL && temp->data != key) {

            prev = temp;

            temp = temp->next;

        }

        // If the key is not present

        if (temp == NULL) return

        // Remove the node

        prev->next = temp->next;
```

```
    }

    int searchlist(int key){

        struct node *temp = head;

        while(temp != NULL) {

            if (temp->data == key) {

                temp=temp->next;

                return 1;

            } else

                return 0;

        }

        return key;

    }

    int main(){

        int k=0;

        insertatbegin(12);

        insertatbegin(22);

        insertatend(30);

        insertatend(44);
```



```
insertatbegin(50);

insertafternode(head->next->next, 33);

cout << "Linked List: ";

// print list

printList();

deleteatbegin();

deleteatend();

deletenode(12);

cout << "\nLinked List after deletion: ";

// print list

printList();

insertatbegin(4);

insertatbegin(16);

cout << "\nUpdated Linked List: ";

printList();

k = searchlist(16);

if (k == 1)

    cout << "\nElement is found";
```

```
else

    cout << "\nElement is not present in the list";

return 0;

}
```

Output

```
Linked List:
[ 50  22  12  33  30  44 ]
Linked List after deletion:
[ 22  33  30 ]
Updated Linked List:
[ 16  4  22  33  30 ]
Element is found
```

❖ Operations on Doubly Linked List

- I. **Insertion** – Adds an element at the beginning of the list.
- II. **Insert Last** – Adds an element at the end of the list.
- III. **Insert After** – Adds an element after an item of the list.
- IV. **Deletion** – Deletes an element at the beginning of the list.
- V. **Delete Last** – Deletes an element from the end of the list.
- VI. **Delete** – Deletes an element from the list using the key.
- VII. **Display forward** – Displays the complete list in a forward manner.
- VIII. **Display backward** – Displays the complete list in a backward manner.

Algorithm

```
1. START
2. Create a new node with three variables: prev, data, next.
3. Store the new data in the data variable
4. If the list is empty, make the new node as head.
5. Otherwise, link the address of the existing first node to the
   next variable of the new node, and assign null to the prev variable.
6. Point the head to the new node.
7. END
```

Algorithm

```
1. START
2. Check the status of the doubly linked list
3. If the list is empty, deletion is not possible
4. If the list is not empty, the head pointer is
   shifted to the next node.
5. END
```

Complete Implementation

```
#include <iostream>
#include <cstring>
#include <cstdlib>
#include <cstdbool>
using namespace std;
struct node {
    int data;
    int key;
    struct node *next;
    struct node *prev;
};
```

```
//this link always point to first Link
struct node *head = NULL;
//this link always point to last Link
struct node *last = NULL;
struct node *current = NULL;
//is list empty
bool isEmpty(){
    return head == NULL;
}
//display the list in from first to last
void displayForward(){
    //start from the beginning
    struct node *ptr = head;
    //navigate till the end of the list
    cout << "\n[ ";
    while(ptr != NULL) {
        cout << "(" << ptr->key << "," << ptr->data << ")";
        ptr = ptr->next;
    }
    cout << "]" << endl;
}
//display the list from last to first
void displayBackward(){
    //start from the last
    struct node *ptr = last;
    //navigate till the start of the list
    cout << "\n[ ";
    while(ptr != NULL) {
```

```
//print data
cout << "(" << ptr->key << "," << ptr->data << ")";

//move to next item
ptr = ptr ->prev;
cout << " ";
}
cout << "]" << endl;
}

//insert link at the first location
void insertFirst(int key, int data){
    //create a link
    struct node *link = (struct node*) malloc(sizeof(struct node));
    link->key = key;
    link->data = data;
    if(isEmpty()) {
        //make it the last link
        last = link;
    } else {
        //update first prev link
        head->prev = link;
    }
    //point it to old first link
    link->next = head;
    //point first to new first link
    head = link;
}

//insert link at the last location
void insertLast(int key, int data){
```

```
//create a link
struct node *link = (struct node*) malloc(sizeof(struct node));
link->key = key;
link->data = data;
if(isEmpty()) {
    //make it the last link
    last = link;
} else {
    //make link a new last link
    last->next = link;
    //mark old last node as prev of new link
    link->prev = last;
}
//point last to new last node
last = link;
}

//delete first item
struct node* deleteFirst(){
    //save reference to first link
    struct node *tempLink = head;
    //if only one link
    if(head->next == NULL) {
        last = NULL;
    } else {
        head->next->prev = NULL;
    }
    head = head->next;
```

```
//return the deleted link
return tempLink;
}

//delete link at the last location
struct node* deleteLast(){
    //save reference to last link
    struct node *tempLink = last;

    //if only one link
    if(head->next == NULL) {
        head = NULL;
    } else {
        last->prev->next = NULL;
    }
    last = last->prev;

    //return the deleted link
    return tempLink;
}

//delete a link with given key
struct node* deletenode(int key){
    //start from the first link
    struct node* current = head;
    struct node* previous = NULL;

    //if list is empty
    if(head == NULL) {
        return NULL;
    }

    //navigate through list
    while(current->key != key) {
```

```
//if it is last node
if(current->next == NULL) {
    return NULL;
} else {
    //store reference to current link
    previous = current;
    //move to next link
    current = current->next;
}
}
//found a match, update the link
if(current == head) {
    //change first to point to next link
    head = head->next;
} else {
    //bypass the current link
    current->prev->next = current->next;
}
if(current == last) {
    //change last to point to prev link
    last = current->prev;
} else {
    current->next->prev = current->prev;
}
return current;
}
bool insertAfter(int key, int newKey, int data){
```



```
//start from the first link
struct node *current = head;
//if list is empty
if(head == NULL) {
    return false;
}
//navigate through list
while(current->key != key) {
    //if it is last node
    if(current->next == NULL) {
        return false;
    } else {
        //move to next link
        current = current->next;
    }
}
//create a link
struct node *newLink = (struct node*) malloc(sizeof(struct node));
newLink->key = key;
newLink->data = data;
if(current == last) {
    newLink->next = NULL;
    last = newLink;
} else {
    newLink->next = current->next;
    current->next->prev = newLink;
}
newLink->prev = current;
current->next = newLink;
```

```
        return true;
    }
int main(){
    insertFirst(1,10);
    insertFirst(2,20);
    insertFirst(3,30);
    insertFirst(4,1);
    insertFirst(5,40);
    insertFirst(6,56);
    printf("\nList (First to Last): ");
    displayForward();
    printf("\n");
    printf("\nList (Last to first): ");
    displayBackward();
    printf("\nList , after deleting first record: ");
    deleteFirst();
    displayForward();
    printf("\nList , after deleting last record: ");
    deleteLast();
    displayForward();
    printf("\nList , insert after key(4) : ");
    insertAfter(4, 7, 13);
    displayForward();
    printf("\nList , after delete key(4) : ");
    deletenode(4);
    displayForward();
    return 0;
}
```

Output

List (First to Last):

[(6, 56) (5, 40) (4, 1) (3, 30) (2, 20) (1, 10)]

List (Last to First):

[(1, 10) (2, 20) (3, 30) (4, 1) (5, 40) (6, 56)]

List, after deleting first record:

[(5, 40) (4, 1) (3, 30) (2, 20) (1, 10)]

List, after deleting last record:

[(5, 40) (4, 1) (3, 30) (2, 20)]

List, insert after key(4):

[(5, 40) (4, 1) (7, 13) (3, 30) (2, 20)]

List, after delete key(4):

[(5, 40) (7, 13) (3, 30) (2, 20)]

❖ Operations on Circular Linked List

- i. **insert** – Inserts an element at the start of the list.

Algorithm

1. START
2. Check if the list is empty
3. If the list is empty, add the node and point the head to this node
4. If the list is not empty, link the existing head as the next node to the new node.
5. Make the new node as the new head.
6. END

```
#include <iostream>

#include <cstring>

#include <cstdlib>

#include <cstdbool>

struct node {

    int data;

    int key;

    struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

bool isEmpty(){

    return head == NULL;

}

//insert link at the first location

void insertFirst(int key, int data){

    //create a link

    struct node *link = (struct node*) malloc(sizeof(struct node));
```

```
link->key = key;

link->data = data;

if (isEmpty()) {

    head = link;

    head->next = head;

} else {

    //point it to old first node

    link->next = head;

    //point first to new first node

    head = link;

}

}

//display the list

void printList(){

    struct node *ptr = head;

    printf("\n[ ");

    //start from the beginning

    if(head != NULL) {
```

```
while(ptr->next != ptr) {  
  
    printf("(%d,%d) ",ptr->key,ptr->data);  
  
    ptr = ptr->next;  
  
}  
  
}  
  
printf(" ]");  
  
}  
  
int main(){  
  
    insertFirst(1,10);  
  
    insertFirst(2,20);  
  
    insertFirst(3,30);  
  
    insertFirst(4,1);  
  
    insertFirst(5,40);  
  
    insertFirst(6,56);  
  
    printf("Circular Linked List: ");  
  
    //print list  
  
    printList();  
  
    return 0;
```

```
}
```

Output

Circular Linked List:

[(6,56) (5,40) (4,1) (3,30) (2,20)]

- ii. **delete** – Deletes an element from the start of the list.

Algorithm

1. START
2. If the list is empty, then the program is returned.
3. If the list is not empty, we traverse the list using a current pointer that is set to the head pointer and create another pointer previous that points to the last node.
4. Suppose the list has only one node, the node is deleted by setting the head pointer to NULL.
5. If the list has more than one node and the first node is to be deleted, the head is set to the next node and the previous is linked to the new head.
6. If the node to be deleted is the last node, link the preceding node of the last node to head node.
7. If the node is neither first nor last, remove the node by linking its preceding node to its succeeding node.
8. END

```
#include <iostream>
```

```
#include <cstring>
```

```
#include <cstdlib>
```

```
#include <stdbool>
```

```
struct node {  
  
    int data;  
  
    int key;  
  
    struct node *next;  
  
};  
  
struct node *head = NULL;  
  
struct node *current = NULL;  
  
bool isEmpty(){  
  
    return head == NULL;  
  
}  
  
//insert link at the first location  
  
void insertFirst(int key, int data){  
  
    //create a link  
  
    struct node *link = (struct node*) malloc(sizeof(struct node));  
  
    link->key = key;  
  
    link->data = data;  
  
    if (isEmpty()) {  
  
        head = link;
```



```
        head->next = head;

    } else {

        //point it to old first node

        link->next = head;

        //point first to new first node

        head = link;

    }

}

//delete first item

struct node * deleteFirst(){

    //save reference to first link

    struct node *tempLink = head;

    if(head->next == head){

        head = NULL;

        return tempLink;

    }

    //mark next to first link as first

    head = head->next;
```

```
//return the deleted link

return tempLink;

}

//display the list

void printList(){

    struct node *ptr = head;

    //start from the beginning

    if(head != NULL) {

        while(ptr->next != ptr) {

            printf("(%d,%d) ",ptr->key,ptr->data);

            ptr = ptr->next;

        }

    }

}

int main(){

    insertFirst(1,10);

    insertFirst(2,20);

    insertFirst(3,30);
```

```
insertFirst(4,1);

insertFirst(5,40);

insertFirst(6,56);

printf("Circular Linked List: ");

//print list

printList();

deleteFirst();

printf("\nList after deleting the first item: ");

printList();

return 0;

}
```

Output

```
Circular Linked List: (6,56) (5,40) (4,1) (3,30) (2,20)
List after deleting the first item: (5,40) (4,1) (3,30) (2,20)
```

- iii. **display** – Displays the list.

Algorithm

1. START
2. Walk through all the nodes of the list and print them
3. END

```
#include <iostream>

#include <cstring>

#include <cstdlib>

#include <cstdbool>

struct node {

    int data;

    int key;

    struct node *next;

};

struct node *head = NULL;

struct node *current = NULL;

bool isEmpty(){

    return head == NULL;

}

//insert link at the first location

void insertFirst(int key, int data){

    //create a link

    struct node *link = (struct node*) malloc(sizeof(struct node));
```

```
link->key = key;

link->data = data;

if (isEmpty()) {

    head = link;

    head->next = head;

} else {

    //point it to old first node

    link->next = head;

    //point first to new first node

    head = link;

}

}

//display the list

void printList(){

    struct node *ptr = head;

    printf("\n[ ");

    //start from the beginning

    if(head != NULL) {
```

```
while(ptr->next != ptr) {  
  
    printf("(%d,%d) ",ptr->key,ptr->data);  
  
    ptr = ptr->next;  
  
}  
  
}  
  
printf(" ]");  
  
}  
  
int main(){  
  
    insertFirst(1,10);  
  
    insertFirst(2,20);  
  
    insertFirst(3,30);  
  
    insertFirst(4,1);  
  
    insertFirst(5,40);  
  
    insertFirst(6,56);  
  
    printf("Circular Linked List: ");  
  
    //print list  
  
    printList();  
  
    return 0;
```

```
}
```

Output

Circular Linked List:

[(6,56) (5,40) (4,1) (3,30) (2,20)]

❖ Representation of Stack and Queue using Linked List

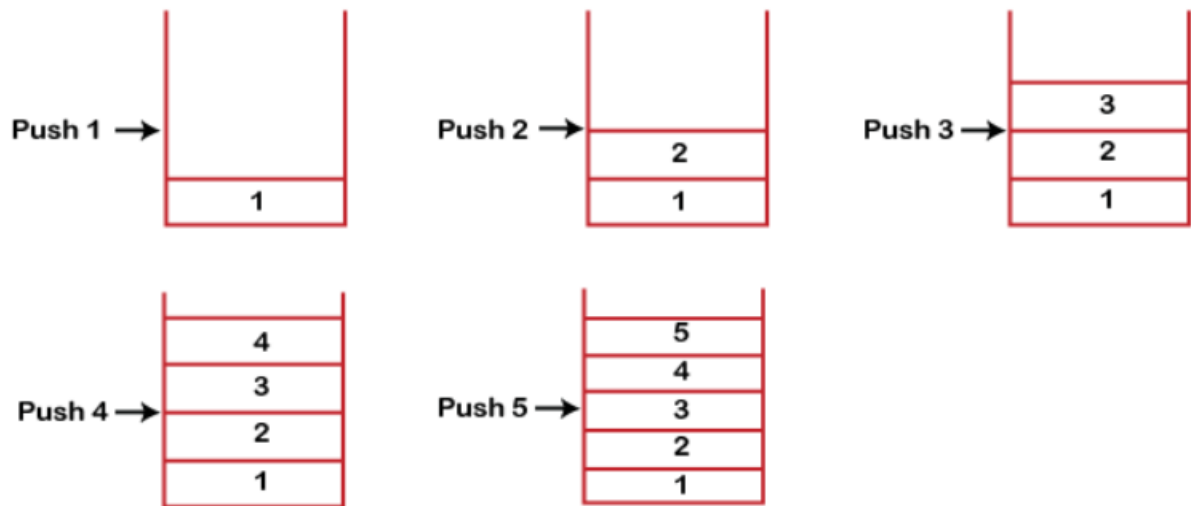
Q. What is a Stack?

A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.

Stack has one end, whereas the Queue has two ends (**front and rear**). It contains only one pointer **top pointer** pointing to the topmost element of the stack. Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack. In other words, a *stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.*

Some key points related to stack

- i. It is called as stack because it behaves like a real-world stack, piles of books, etc.
- ii. A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- iii. It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.



Standard Stack Operations

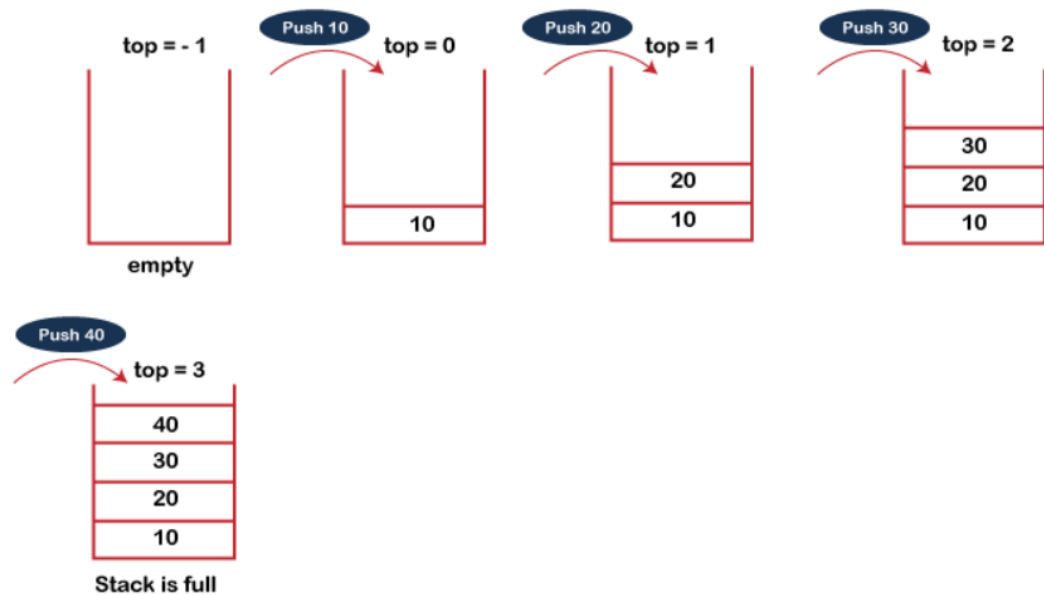
The following are some common operations implemented on the stack:

A. push(): When we insert an element in a stack then the operation is known as a push.

If the stack is full then the overflow condition occurs.

- i. Before inserting an element in a stack, we check whether the stack is full.
- ii. If we try to insert the element in a stack, and the stack is full, then the *overflow* condition occurs.
- iii. When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- iv. When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the **top**.

- v. The elements will be inserted until we reach the *max* size of the stack.

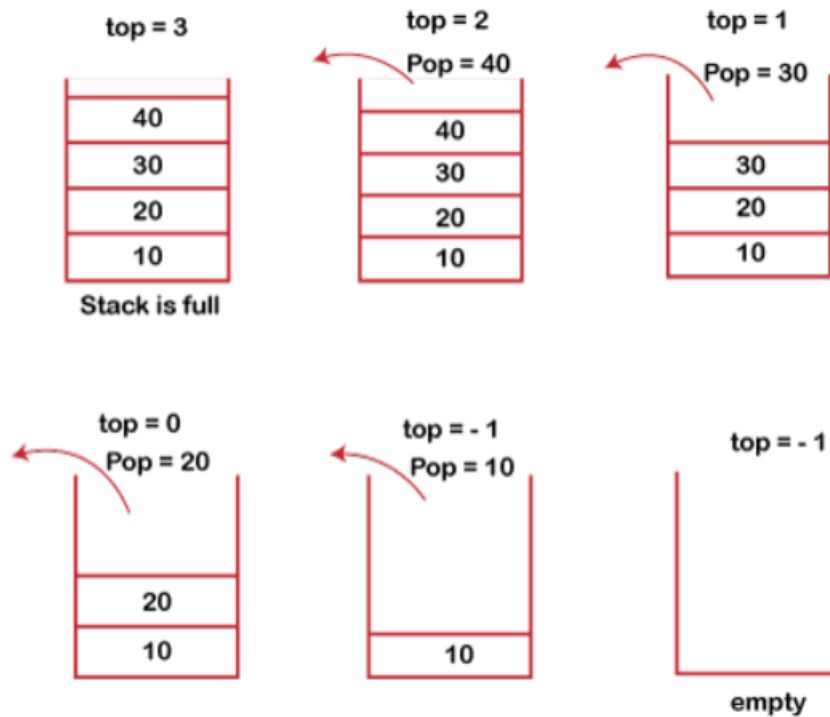


B. pop(): When we delete an element from the stack, the operation is known as a pop. If

the stack is empty means that no element exists in the stack, this state is known as an underflow state.

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the *underflow* condition occurs.
- If the stack is not empty, we first access the element which is pointed by the *top*

- iv. Once the pop operation is performed, the top is decremented by 1,
i.e., **top=top-1**.



- C. isEmpty():** It determines whether the stack is empty or not.
- D. isFull():** It determines whether the stack is full or not.'
- E. peek():** It returns the element at the given position.
- F. count():** It returns the total number of elements available in a stack.
- G. change():** It changes the element at the given position.
- H. display():** It prints all the elements available in the stack.

1. Stack Using Linked List

A **Stack** follows the **Last In First Out (LIFO)** principle, meaning the last element added to the stack will be the first one to be removed. Using a linked list to implement a stack allows for dynamic memory allocation, and the stack can grow and shrink as needed.

Operations on Stack

- i. **Push:** Add an element to the top of the stack.
- ii. **Pop:** Remove the top element from the stack.
- iii. **Peek:** Retrieve the top element without removing it.
- iv. **isEmpty:** Check if the stack is empty.

Structure of a Node (in Stack)

```
struct Node {  
  
    int data; // Value of the node  
  
    Node* next; // Pointer to the next node  
  
};
```

A. Push Operation

To push an element onto the stack:

- i. Create a new node.
- ii. Assign its data value.

- iii. Point its next to the current top of the stack.
- iv. Update the top to be this new node.

```
void push(int data) {  
  
    Node* newNode = new Node(); // Step 1  
  
    newNode->data = data;    // Step 2  
  
    newNode->next = top;    // Step 3  
  
    top = newNode;        // Step 4  
  
}
```

B. Pop Operation

- i. Check if the stack is empty.
- ii. Store the current top node.
- iii. Move the top pointer to the next node.
- iv. Delete the old top node.

```
int pop() {  
  
    if (top == nullptr) {  
  
        cout << "Stack is empty!" << endl;  
  
        return -1;  
  
    }  
  
    Node* temp = top;    // Step 2
```

```
top = top->next;    // Step 3

int popped = temp->data;

delete temp;        // Step 4

return popped;

}
```

Example of Stack using Linked List

```
#include<iostream>

using namespace std;

struct Node {

    int data;

    Node* next;

};

class Stack{

private:

    Node* top;

public:

    Stack() { top = nullptr; }

    void push(int data) {

        Node* newNode = new Node();

        newNode->data = data;

        newNode->next = top;

        top = newNode;
```

```
        cout << data << " pushed to stack\n";
    }

    int pop() {
        if (top == nullptr) {
            cout << "Stack is empty\n";
            return -1;
        }

        Node* temp = top;
        top = top->next;
        int popped = temp->data;
        delete temp;
        return popped;
    }

    bool isEmpty() {
        return top == nullptr;
    }

    void peek() {
        if (top != nullptr)
            cout << "Top element is " << top->data << endl;
        else
            cout << "Stack is empty\n";
    }
}
```

```
};  
  
int main() {  
  
    Stack s;  
  
    s.push(10);  
  
    s.push(20);  
  
    s.push(30);  
  
    s.peak();  
  
    cout << s.pop() << " popped from stack\n";  
  
    s.peak();  
  
    return 0;  
  
}
```

Q. What is Queue in Data Structures?

A queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle.

It operates like a line where elements are added at one end (**rear**) and removed from the other end (**front**).

Basic Operations of Queue Data Structure

- i. **Enqueue (Insert):** Adds an element to the rear of the queue.
- ii. **Dequeue (Delete):** Removes and returns the element from the front of the queue.
- iii. **Peek:** Returns the element at the front of the queue without removing it.
- iv. **Empty:** Checks if the queue is empty.
- v. **Full:** Checks if the queue is full.

Applications of Queue

- i. **Task scheduling** in operating systems
- ii. **Data transfer** in network communication
- iii. **Simulation** of real-world systems (e.g., waiting lines)
- iv. **Priority queues** for event processing queues for event processing

2. Queue Using Linked List

A **Queue** follows the **First In First Out (FIFO)** principle, meaning the first element added to the queue will be the first one to be removed. In a queue implemented using a linked list, we maintain two pointers: **front** and **rear**.

Operations on Queue

- i. **Enqueue**: Add an element to the rear of the queue.
- ii. **Dequeue**: Remove an element from the front of the queue.
- iii. **isEmpty**: Check if the queue is empty.
- iv. **Peek**: Retrieve the front element without removing it.

```
struct Node {  
  
    int data;    // Value of the node  
  
    Node* next;  // Pointer to the next node  
  
};
```


Enqueue Operation

To add an element to the queue:

1. Create a new node.
2. If the queue is empty, set both front and rear to this new node.
3. Otherwise, point the current rear node's next to the new node and update rear.

```
void enqueue(int data) {
```

```
    Node* newNode = new Node(); // Step 1
```

```
    newNode->data = data;
```

```
    newNode->next = nullptr;
```

```
    if (rear == nullptr) { // Step 2
```

```
        front = rear = newNode;
```

```
        return;
```

```
    }
```

```
    rear->next = newNode; // Step 3
```

```
    rear = newNode;
```

```
}
```

Dequeue Operation

To remove an element from the queue:

1. Check if the queue is empty.
2. Store the current front node.
3. Move the front pointer to the next node.

4. If front becomes nullptr, set rear to nullptr (queue is now empty).
5. Delete the old front node.

```
int dequeue() {  
  
    if (front == nullptr) {  
  
        cout << "Queue is empty!\n";  
  
        return -1;  
  
    }  
  
    Node* temp = front; // Step 2  
  
    front = front->next; // Step 3  
  
    if (front == nullptr) // Step 4  
  
        rear = nullptr;  
  
    int data = temp->data;  
  
    delete temp; // Step 5  
  
    return data;  
  
}
```

Example of Queue using Linked List

```
#include<iostream>  
  
using namespace std;  
  
struct Node {  
  
    int data;  
  
    Node* next;  
  
};
```

```
class Queue {  
  
private:  
  
    Node* front, * rear;  
  
public:  
  
    Queue() { front = rear = nullptr; }  
  
    void enqueue(int data) {  
  
        Node* newNode = new Node();  
  
        newNode->data = data;  
  
        newNode->next = nullptr;  
  
        if (rear == nullptr) {  
  
            front = rear = newNode;  
  
            return;  
  
        }  
  
        rear->next = newNode;  
  
        rear = newNode;  
  
        cout << data << " enqueued to queue\n";  
  
    }  
  
    int dequeue() {  
  
        if (front == nullptr) {  
  
            cout << "Queue is empty\n";  
  
            return -1;  
  
        }  
  
    }
```

```
Node* temp = front;

front = front->next;

if (front == nullptr)

    rear = nullptr;

int data = temp->data;

delete temp;

return data;
}

bool isEmpty() {

    return front == nullptr;

}

void peek() {

    if (front != nullptr)

        cout << "Front element is " << front->data << endl;

    else

        cout << "Queue is empty\n";

}

};

int main() {

    Queue q;

    q.enqueue(10);

    q.enqueue(20);
```

```
q.enqueue(30);  
  
q.peek();  
  
cout << q.dequeue() << " dequeued from queue\n";  
  
q.peek();  
  
return 0;  
  
}
```